MODULE 19 · DOMAIN 4 - MONITORING AND LOGGING

Tracing and Distributed Observability

Following one request across many services: X-Ray, the move to OpenTelemetry, Transaction Search, Application Signals and SLOs, and managed Prometheus and Grafana.

DOMAIN 4 of 6 EXAM WEIGHT 15% MODULE 19 of 34

admin@sthithapragna.com

Metrics say something is **wrong**. Logs say **what** happened. Traces say **where** in the chain it happened.

One request, one trace

01

API Gateway

Starts the trace and adds a trace header.

02

Lambda

Records its own segment, same trace ID.

03

DynamoDB call

A subsegment: how long the query took.

04

Partner API

A subsegment: 1.8 seconds. There it is.

05

Service map

Every hop, drawn with latency and errors.

ON THE EXAM

The trace ID must travel with the request. A service that drops the header breaks the trace in two.

Tracing vocabulary

TERM	MEANING
Trace	Everything that happened for one request, end to end
Segment	One service's part of the trace (X-Ray term)
Subsegment	A call that service made: a database, an HTTP API
Span	The OpenTelemetry term for a segment or subsegment
Trace context	The header carrying the trace ID between services
Trace map	The graph of services and their connections

ON THE EXAM

X-Ray says segment; OpenTelemetry says span. Exam options may use either.

The catalogue has moved: X-Ray instrumentation

DATE	CHANGE
November 2024	X-Ray accepts OpenTelemetry directly (OTLP endpoint); Transaction Search launched
25 February 2026	X-Ray SDKs and daemon enter maintenance mode: security fixes only
25 February 2027	X-Ray SDKs and daemon reach end of support
Ongoing	The X-Ray service itself continues, with new features

THE RULE

New instrumentation should use OpenTelemetry. Existing X-Ray SDK traces are still accepted and processed.

X-Ray SDK, or OpenTelemetry?

OPENTELEMETRY

The current path

- Industry standard, many languages
- Zero-code auto-instrumentation
- Collector: ADOT or the CloudWatch agent
- Sends to X-Ray, CloudWatch, or elsewhere

X-RAY SDK AND DAEMON

In maintenance mode

- AWS-specific libraries
- Security fixes only since February 2026
- End of support February 2027
- Still valid in exam-guide answers

THE TIEBREAKER

If an option uses the X-Ray SDK, it reflects the guide and still works. For new, future-proof work, choose OpenTelemetry.

AWS Distro for OpenTelemetry

- **ADOT** is AWS's supported distribution of the OpenTelemetry SDKs and collector.
- **Auto-instrumentation** agents trace common libraries with no code changes.
- Runs as a Lambda layer, an ECS sidecar, an EKS add-on, or on EC2.
- One collector can send traces to X-Ray and metrics to CloudWatch or managed Prometheus.

> ON THE EXAM
>
> The CloudWatch agent can also receive OpenTelemetry data, so one agent can cover metrics, logs and traces.

Turning on tracing, by platform

PLATFORM	HOW
API Gateway	Enable tracing on the stage
Lambda	Active tracing, or the ADOT Lambda layer
ECS	A collector sidecar in the task, plus instrumented code
EKS	The ADOT add-on or CloudWatch agent, often as a DaemonSet
EC2	The CloudWatch agent or ADOT collector on the instance

ON THE EXAM

Traces stop at a hop that is not instrumented. Every service in the chain must pass the context on.

Sampling

- Tracing every request is expensive, so **sampling rules** choose which to trace.
- A rule has a **reservoir**, a fixed number per second, and a **rate**, a percentage beyond it.
- The default traces the first request each second, then 5% of the rest.
- Rare failures may never be sampled. Raise sampling for important paths, or use Transaction Search.

> ON THE EXAM
>
> Rules are managed centrally in X-Ray, so sampling changes need no redeployment.

Annotations or metadata?

ANNOTATIONS

Indexed, searchable

- Simple key-value pairs
- Indexed for filter expressions
- Find traces by order ID or customer tier
- Limited number per trace

METADATA

Stored, not searchable

- Any object, including nested data
- Not indexed
- Useful context once a trace is found
- Cannot filter on it

THE TIEBREAKER

'Find all traces for customer tier gold' needs an annotation. Metadata cannot be searched.

Groups and Insights

- A **group** is a saved filter expression, such as all traces for the checkout API.
- Each group gets its own metrics and trace map, so you can alarm per group.
- **X-Ray Insights** detects anomalies in a group, such as rising faults, and tracks them as incidents.
- Insights events can go to EventBridge, starting notifications or remediation.

> ON THE EXAM
>
> Be told automatically when a service's fault rate becomes abnormal, with root-cause context: X-Ray Insights.

Transaction Search

- Sends **every span** to CloudWatch Logs, in the aws/spans log group, without sampling limits.
- Search and analyse spans with Logs Insights, by any attribute.
- A chosen percentage is indexed as trace summaries for the X-Ray trace views.
- Works with spans from OpenTelemetry, ADOT, or the X-Ray SDK.

> ON THE EXAM
>
> Find the one failing request among millions, even if it was never sampled: Transaction Search.

CloudWatch Application Signals

- An APM experience: services discovered automatically, with dependencies drawn.
- Standard health metrics for each operation: volume, latency, faults and errors.
- Auto-instrumentation on EKS, ECS, Lambda and EC2 for Java, Python, Node.js and .NET.
- **Service level objectives** tracked for each service or operation.

> ON THE EXAM
>
> Application performance monitoring on AWS with least set-up: Application Signals.

SLO vocabulary

TERM	MEANING	EXAMPLE
SLI	What you measure	Share of requests under 300 ms
SLO	The target for it, over a window	99.9% over 30 days
Error budget	Failure the SLO allows	About 43 minutes a month
Burn rate	How fast the budget is being spent	14x means gone in about 2 days

ON THE EXAM

Alarm on burn rate, not on single errors: it warns while there is still budget left to protect.

Amazon Managed Service for Prometheus

- A managed, Prometheus-compatible store for metrics, queried with **PromQL**.
- Data arrives by remote write, from Prometheus or ADOT, or from an **agentless managed collector** for EKS.
- Recording and alerting rules run in the workspace; alerts route through Alertmanager to SNS.
- Suits teams already using Prometheus, especially with Kubernetes.

> ON THE EXAM
>
> Keep Prometheus and PromQL, stop running Prometheus servers: managed Prometheus.

Amazon Managed Grafana

- Managed Grafana workspaces: no servers, patching or scaling to run.
- Sign-in through **IAM Identity Center** or a **SAML** identity provider, with Admin, Editor and Viewer roles.
- Data sources include CloudWatch, managed Prometheus, X-Ray, OpenSearch and Athena.
- Existing Grafana dashboards can be imported.

> ON THE EXAM
>
> Grafana dashboards, corporate sign-in, no Grafana servers: Managed Grafana with IAM Identity Center or SAML.

CloudWatch-native, or Prometheus and Grafana?

PROMETHEUS AND GRAFANA

Open-source ecosystem

- Teams already on Prometheus and PromQL
- Kubernetes-heavy estates
- Existing Grafana dashboards
- Portable across clouds

CLOUDWATCH AND APPLICATION SIGNALS

AWS-native

- No new query languages
- Deep integration with AWS services
- Alarms, SLOs and events built in
- Least to learn for AWS-first teams

THE TIEBREAKER

The deciding clue is usually what the team already uses. Migrations with least change keep the existing tools, managed.

> A checkout flow runs as eight microservices on ECS with Fargate. Latency has doubled, but each service's CPU, memory and logs look normal, and nobody can tell which service is slow.
>
> The team wants to find the slow hop, using instrumentation that will remain supported long term.

What should the DevOps engineer do?

A. Add the X-Ray SDK to every service and run the X-Ray daemon as a sidecar

B. Use ADOT auto-instrumentation in each service, with a collector sidecar sending traces to X-Ray

C. Enable Container Insights with enhanced observability

D. Increase every service's log level to DEBUG

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A X-Ray SDK and daemon

Would find the slow hop, but uses libraries in maintenance mode since February 2026. Fails 'supported long term'.

B ADOT and a collector sidecar

Traces every hop to X-Ray with OpenTelemetry, the supported path.

C Container Insights

Per-container resource metrics, which already look normal. Does not follow requests.

D DEBUG logging

More detail inside each service, still no view across them.

THE SIGNAL

Metrics and logs normal, slow somewhere across services: tracing. 'Supported long term': OpenTelemetry.

> An e-commerce team traces requests with X-Ray. Support staff often need to find the trace for a specific order ID, or all traces for premium customers.
>
> Today they search by time and guess. The team wants to filter traces directly by order ID and customer tier.

What should the developers add to their instrumentation?

A Record the order ID and tier as trace metadata

B Write the order ID and tier to CloudWatch Logs

C Record the order ID and tier as annotations, and use filter expressions

D Name a subsegment after each order ID

◆ PAUSE AND DECIDE

What should the developers add to their instrumentation?

A **Metadata**

Stored with the trace but not indexed. Filter expressions cannot use it.

B **Logs**

Searchable in logs, but does not filter traces.

C **Annotations**

Indexed key-value pairs: filter expressions find traces by order ID or tier.

D **Subsegment names**

Creates a new name per order, cluttering maps, and is not how traces are searched.

THE SIGNAL

Find traces by a business value: annotations. Extra context you only read: metadata.

> A payments service is traced with the default X-Ray sampling. A rare failure affects a handful of requests each day, and those requests are almost never sampled.
>
> Engineers need to search all spans for any request, sampled or not, and analyse them with Logs Insights.

What should the DevOps engineer enable?

A CloudWatch Transaction Search

B An X-Ray group for the payments service

C An annotation for payment status

D X-Ray Insights on the default group

◆ PAUSE AND DECIDE

What should the DevOps engineer enable?

A Transaction Search

Every span goes to CloudWatch Logs, searchable with Logs Insights, whether sampled or not.

B A group

Filters traces that exist. Unsampled requests are still missing.

C An annotation

Makes sampled traces searchable. Unsampled ones still do not exist.

D X-Ray Insights

Detects anomalies in sampled data. A handful of requests will not move it.

THE SIGNAL

Rare requests missing from traces because of sampling, analysed with Logs Insights: Transaction Search.

> A product team commits to 99.9% availability for its checkout API over each 30-day period.
>
> It wants to track that objective continuously, and to be alerted early when the rate of failures threatens it, not on every individual error.

What should the DevOps engineer set up?

A. A static alarm that fires whenever the 5xx count exceeds zero

B. A CloudWatch dashboard showing daily availability

C. A Synthetics canary calling the API every minute

D. An Application Signals SLO for the operation, with a burn-rate alarm

◆ PAUSE AND DECIDE

What should the DevOps engineer set up?

A **Alarm on any error**

Fires on every error. the exact noise the team wants to avoid.

B **Dashboard**

Shows the objective, but alerts nobody.

C **Canary**

Measures synthetic requests, not the objective over real traffic.

D **SLO with burn-rate alarm**

Tracks the 30-day objective and warns when the budget is burning too fast.

THE SIGNAL

An availability objective over a window, alerting on threat rather than noise: an SLO and a burn-rate alarm.

A platform team runs self-managed Prometheus and Grafana on its EKS clusters. Keeping them patched, scaled and highly available takes too much effort.

The team wants managed services, keeping its PromQL queries and existing dashboards, with the fewest changes.

What should the platform team do?

A Move to Container Insights, and rebuild every dashboard in CloudWatch

B Amazon Managed Service for Prometheus with a managed collector, and Amazon Managed Grafana with the dashboards imported

C Send metrics to Amazon OpenSearch Service

D Send traces to X-Ray and use its trace map instead

◆ PAUSE AND DECIDE

What should the platform team do?

A Container Insights and rebuild

Managed, but abandons PromQL and every dashboard. Many changes.

B Managed Prometheus and Grafana

Same query language, same dashboards; the servers become AWS's problem.

C OpenSearch

A different query model, and still dashboards to rebuild.

D X-Ray

Traces, not the metrics and dashboards the team relies on.

THE SIGNAL

Keep the tools, lose the servers: managed Prometheus and managed Grafana.

A company adopts Amazon Managed Grafana. Engineers already sign in to AWS through IAM Identity Center, connected to the corporate directory.

Grafana access must use the same corporate identities, with SRE staff as editors and everyone else as viewers.

How should the DevOps engineer configure authentication?

A Create an IAM user for each engineer and share Grafana credentials

B Set up an Amazon Cognito user pool for the workspace

C Use IAM Identity Center authentication for the workspace, assigning groups to Editor and Viewer roles

D Create Grafana API keys for each engineer

◆ PAUSE AND DECIDE

How should the DevOps engineer configure authentication?

A **IAM users**

Separate long-lived identities, and not how Managed Grafana signs users in.

B **Cognito**

A second, separate identity store. Managed Grafana supports Identity Center or SAML.

C **IAM Identity Center groups to roles**

Same corporate sign-in, with groups mapped to Grafana's Editor and Viewer roles.

D **API keys**

Meant for automation, not people signing in.

THE SIGNAL

Managed Grafana users sign in with IAM Identity Center or SAML. Map groups to Admin, Editor, Viewer.

Module 19 in five lines

- Metrics say something is wrong; logs say what; traces say where across services.
- X-Ray SDKs and daemon: maintenance since February 2026. New work uses OpenTelemetry via ADOT.
- Annotations are searchable; metadata is not. Sampling can miss rare requests.
- Transaction Search captures every span; Application Signals tracks SLOs with burn-rate alarms.
- Prometheus users: managed Prometheus and Grafana, with Identity Center or SAML sign-in.

ON THE EXAM

Next, module 20: health checks and self-healing, the last module of Domain 4.